

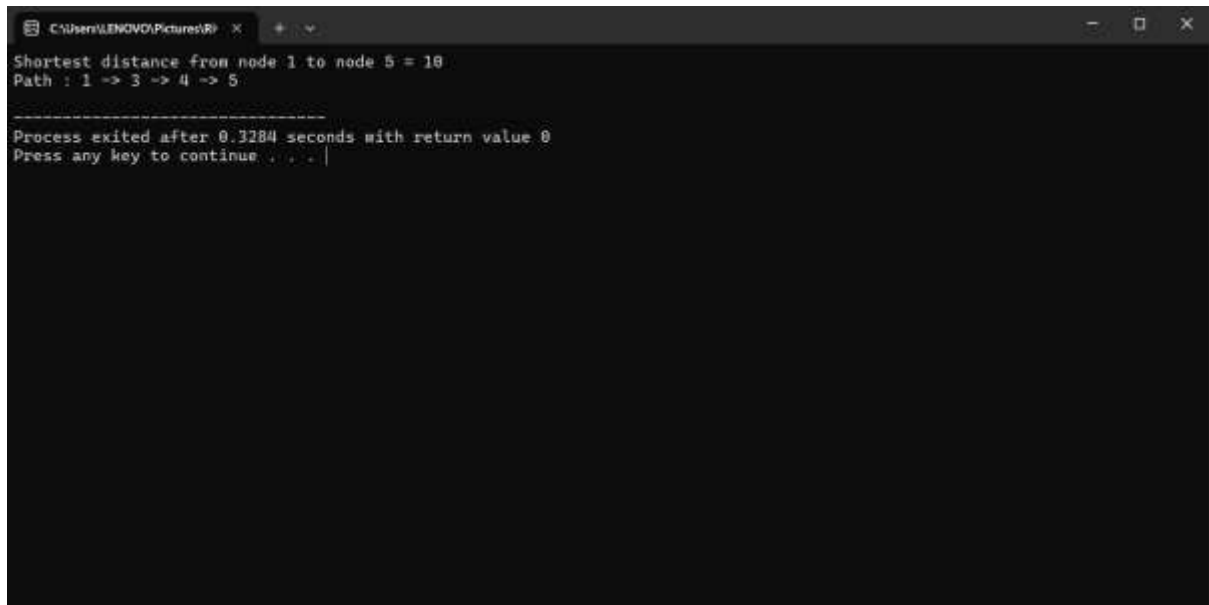
Nama : Rival Mauli Prasetya

NPM : 252310009

Kelas : TI – 25 – PA2

TUGAS 12

Link Github : https://github.com/rivalmauli910-ctrl/252310009_LabAlgoritma.git



```
C:\Users\LENOVO\Pictures\RP >
Shortest distance from node 1 to node 5 = 18
Path : 1 -> 3 -> 4 -> 5

-----
Process exited after 0.3284 seconds with return value 0
Press any key to continue . . . .|
```

TUGAS 12 RivalMP.cpp

```

1  #include <iostream>
2  #include <limits>
3  using namespace std;
4
5  const int V = 6;
6
7  // Fungsi untuk mencari node dengan jarak minimum
8  int minDistance(int dist[], bool visited[])
9  {
10     int min = INT_MAX;
11     int min_index;
12
13     for (int v = 0; v < V; v++)
14     {
15         if (!visited[v] && dist[v] < min)
16         {
17             min = dist[v];
18             min_index = v;
19         }
20     }
21
22     return min_index;
23 }
24
25 // Menampilkan path
26 void printPath(int parent[], int j)
27 {
28     if (parent[j] == -1)
29     {
30         cout << j;
31         return;
32     }
33
34     printPath(parent, parent[j]);
35     cout << " -> " << j;
36 }
37

```

TUGAS 12 RivalMP.cpp

```

47     dist[i] = INT_MAX;
48     visited[i] = false;
49     parent[i] = -1;
50 }
51
52 dist[src] = 0;
53
54 for (int count = 0; count < V - 1; count++)
55 {
56     int u = minDistance(dist, visited);
57
58     visited[u] = true;
59
60     for (int v = 0; v < V; v++)
61     {
62         if (!visited[v] &&
63             graph[u][v] &&
64             dist[u] != INT_MAX &&
65             dist[u] + graph[u][v] < dist[v])
66         {
67             parent[v] = u;
68             dist[v] = dist[u] + graph[u][v];
69         }
70     }
71 }
72
73 cout << "Shortest distance from node "
74      << src << " to node "
75      << dest << " = "
76      << dist[dest] << endl;
77
78 cout << "Path : "

```

```

77
78     cout << "Path : ";
79     printPath(parent, dest);
80     cout << endl;
81 }
82
83 int main()
84 {
85     int graph[V][V] =
86     {
87         {0,4,2,0,0,0},
88         {4,0,1,5,0,0},
89         {2,1,0,8,10,0},
90         {0,5,8,0,2,6},
91         {0,0,10,2,0,3},
92         {0,0,0,6,3,0}
93     };
94
95     int source = 1;
96     int destination = 5;
97
98     dijkstra(graph, source, destination);
99
100     return 0;
101 }

```